

Phase B — Critical Error Protection & Automatic Recovery

Connector version: 1.2.11 **Status:** Implemented and unit/feature tested. Live-site behaviours that require a running WordPress install are listed explicitly under *Live-site verification* — they cannot be exercised on the build VM and must be confirmed on a staging site.

1. Problem

If a MarQira-managed website develops a **critical WordPress/PHP error** (a fatal that stops the site from bootstrapping), the MarQira Connector — the very tool that should let us manage or recover the site — can itself stop loading. The site becomes unmanageable exactly when intervention is needed most.

The requirement is **not** to “fix arbitrary PHP errors”. It is to build a **controlled health-check + rollback mechanism** around the specific, known actions that the connector itself performs, so that a managed action which breaks a site is detected and its *own* change reverted — without ever touching unrelated code.

2. Architecture overview

Three cooperating pieces were added to the connector, plus wiring through the API and dashboard:

Component	File	Responsibility
Health checker	includes/class-marqira-health-check.php	Probes whether WordPress bootstraps, the MarQira REST endpoint responds, and the frontend/admin load without a fatal. Classifies the site as up / down / inconclusive.
Recovery manager	includes/class-marqira-recovery.php	Runs <i>before</i> and <i>after</i> every risky managed action. Snapshots state, arms a sentinel, verifies health afterwards, and rolls back the specific change if the site went critical. Handles loop protection, pre-existing-breakage detection

Component	File	Responsibility
Must-use guard	mu-plugins/ marqira-guard.php	and the audit report. Last-resort, dependency-free shutdown handler that loads <i>before</i> regular plugins. If a request dies with a fatal while a sentinel is armed, it emergency-deactivates only the sentinel's named plugin(s) so the next request can boot.
API wiring	apps/api — UpdateCom mandControl ler, Site model, migration, SiteControlle r	Accepts, stores and re-exposes a structured recovery report attached to the connector's ack.
Dashboard	apps/ dashboard — WebsiteData il.tsx, types/index.t s	Renders a clear recovery banner (rolled back & recovered / rolled back but still unhealthy / pre-existing breakage / healthy).

3. The managed-action flow

Every risky action the connector performs (run_core_update, run_all_plugins_update, run_all_themes_update, and the connector self-update run_plugin_update) is now wrapped:

```
begin(action_id, type, targets)
├─ pre-action health check
│   └─ if ALREADY critical → proceed = false, reason = "pre_existing_critical"
│       (report it; do NOT run the action, do NOT roll back)
├─ snapshot the targets (copy plugin/theme dirs to a backup location)
└─ arm the recovery sentinel (option: marqira_recovery_sentinel)
```

... perform the requested action ...

```
finish_and_verify(action_id, type, targets, snapshot)
├─ post-action health check
│   └─ healthy → clean up snapshot, clear sentinel, done (recovered = n/a)
│       └─ critical → loop-protected rollback of THIS action's change
│           then re-verify health
└─ returns { healthy, rolled_back, recovered, detail, health }
```

This maps directly onto the requested desired flow:

1. **Record current state** — snapshot() copies the affected plugin/theme directories.
 2. **Record versions being changed** — the sentinel + audit report store type and targets (plugin basenames / theme stylesheets).
 3. **Create a rollback point where technically possible** — file-level directory backup for plugins/themes.
 4. **Perform the requested action** — unchanged core WordPress upgrade calls.
 5. **Post-update health checks** — Marqira_Health_Check::run().
 6. **Verify WordPress can bootstrap** — wp_bootstrap probe.
 7. **Verify the REST endpoint responds** — rest_endpoint probe against marqira/v1/health-ping.
 8. **Verify frontend/admin don't fatal** — frontend and admin probes.
-

4. Health checks

Marqira_Health_Check::run() performs up to four probes and returns ['healthy' => bool, 'checks' => [...], 'summary' => string].

Probe	What it requests
wp_bootstrap	internal - confirms core constants/functions are present
rest_endpoint	GET marqira/v1/health-ping (permission __return_true)
frontend	home_url()
admin	wp_login_url()

Classification (probe()):

- HTTP **5xx**, or a body containing a known **fatal signature** (Fatal error, There has been a critical error on this website, Parse error, Allowed memory size, Maximum execution time) → **down**.
- A transport-level WP_Error (timeout, DNS, connection refused) → **inconclusive** (we do not treat an unreachable probe as proof the site is broken — that would risk a false rollback).
- HTTP < 500 with no fatal signature → **up**.

The site is deemed **critical** only when a probe is unambiguously down. Inconclusive probes never by themselves trigger a rollback.

A new REST route GET /marqira/v1/health-ping (added in class-marqira-rest-controller.php) returns a tiny JSON ok payload and is intentionally public so the health checker can reach it even before authentication state is established.

5. Rollback matrix

Managed action	On post-action fatal	Fallback if file restore not possible
Plugin update	Restore the previous plugin directory from the snapshot	Deactivate the offending plugin
Theme update	Restore the previous theme directory from the snapshot	Switch to a default theme
Plugin activation	Deactivate the plugin	—
Theme activation	Restore the previously active theme	Switch to a default theme
Core update	Report only — no destructive automatic core rollback	Rely on WordPress-supported recovery; surface the failure in the dashboard

Core is deliberately **report-only**: silently rewriting core files is riskier than the failure itself, so the connector reports the failure and defers to WordPress's own supported recovery mechanisms.

6. Safety requirements — how each is met

Requirement	Implementation
Only roll back known MarQira-managed actions	Rollback is keyed off the sentinel, which is only armed by begin() for a specific action_id/targets. Nothing else is ever touched.
Never blindly modify unrelated plugins/code	emergency_deactivate() and the recovery rollback operate only on the exact basenames/stylesheets recorded in the sentinel/snapshot. Unit test test-guard.php proves unrelated plugins are preserved.
Keep an audit log	marqira_recovery_last_report (last structured report) + marqira_recovery_guard_events (bounded

Requirement	Implementation
	ring buffer of guard actions, max 25). The report is also POSTed to the API and stored on the site.
Avoid rollback loops	MAX_ATTEMPTS = 1 in the recovery manager; the mu-plugin guard marks the sentinel guard_handled so it never acts on the same sentinel twice.
Detect genuine unhealthiness before reverting	Only an unambiguous down classification triggers rollback; inconclusive never does.
Preserve site data / avoid DB restores	Recovery is file-level only (plugin/theme directories) plus option-level activation flags. It never restores or edits database content.
Report failure + recovery clearly in the dashboard	Structured recovery report flows connector → API → dashboard banner.
Pre-existing breakage is not blamed on the action	begin() runs a pre-action health check; if the site is already critical it returns proceed = false, reason = pre_existing_critical, the action is skipped, and the ack reports the pre-existing condition instead of rolling anything back.

7. Resilient connector access — the must-use guard

Question posed: can part of the connector stay usable even when normal WordPress execution fails?

Answer implemented: a **must-use plugin** (mu-plugins/marqira-guard.php), auto-installed by the connector into WPMU_PLUGIN_DIR on activation and kept in sync on admin_init (md5 comparison). Must-use plugins load **before** regular plugins, so the guard's register_shutdown_function handler is already resident before any risky code runs.

If a request then dies with a PHP fatal **and** a sentinel is armed, the guard's shutdown handler:

1. Confirms the error is a hard fatal (E_ERROR|E_PARSE|E_CORE_ERROR|E_COMPILE_ERROR|E_USER_ERROR|E_RECOVERABLE_ERROR).
2. Reads the sentinel and deactivates **only** the plugin(s) it names, by rewriting the active_plugins option directly (it avoids the plugin API because the environment is already fatal).

3. Records a bounded event and marks the sentinel `guard_handled` (loop protection).

Documented limitations (honest)

- The guard **cannot** recover from a fatal that occurs *before* mu-plugins load — a corrupt `wp-config.php`, a broken WordPress **core** file, or a fatal in a database / object-cache **drop-in**. Those are outside any PHP-level guard.
 - It only **deactivates plugins** (the safest data-preserving action available mid-fatal). File-level version restores are performed by the recovery manager while WordPress is still healthy enough to run it; the guard is the last resort.
 - It never restores or modifies **database** content.
 - If WordPress cannot bootstrap at all, the normal connector plugin genuinely cannot continue executing — we do **not** claim otherwise. The mu-plugin guard is the best realistic mechanism within WordPress's architecture, and its scope is limited to the above.
-

8. Data model & API

- New nullable JSON column `sites.update_command_recovery` (migration `2024_01_07_000002_add_update_command_recovery_to_sites.php`), cast to array on the Site model.
- `POST /api/v1/update-command/ack` now accepts an optional recovery (validated nullable|array; a non-array value is rejected 422). It is stored verbatim and reset to null when a new update command is requested.
- The dashboard site status payload re-exposes it as `command.recovery`.

Report shape:

```
{
  "action_id": "cmd-abc-123",
  "type": "update_plugin",
  "rolled_back": true,
  "recovered": true,
  "reason": null,
  "detail": "Update reverted after a critical error; previous version restored.",
  "health": { "healthy": true, "checks": [ ... ], "summary": "..." }
}
```

9. Dashboard presentation

`WebsiteDetail.tsx` renders a `RecoveryBanner` under the update-command status card with four states:

- **Pre-existing critical** (reason === 'pre_existing_critical') → warning banner: the site was already broken; the update was not run.
 - **Rolled back & recovered** (rolled_back && recovered) → success banner: the change caused a fatal, was reverted, and the site is healthy again.
 - **Rolled back but still unhealthy** (rolled_back && recovered === false) → danger banner: manual intervention needed.
 - **Healthy** (status === 'completed') → subtle confirmation that post-update health checks passed.
-

10. Tests

Connector harness (php tests/run.php) — new files, 65 assertions, all passing:

- test-health-check.php (15) — probe classification (up/down/inconclusive) and overall verdict.
- test-recovery.php (30) — sentinel lifecycle, snapshot/restore, refuse-on-pre-existing, healthy path, broke→file-restore recovery, deactivate fallback, loop protection, audit report.
- test-guard.php (20) — is_fatal classification, emergency_deactivate removes only named plugins (unrelated preserved), event log bounded to 25.

The 4 unrelated failures in the full suite (test-heartbeat-cron, test-site-revoked, test-persistent-pairing, test-data-collector) are **pre-existing** harness gaps (they require wp-admin/includes/plugin.php, absent in the standalone harness) and are not related to Phase B.

API feature tests (php artisan test --filter=RemoteUpdateCommandTest) — 17 passing (63 assertions), including 4 new Phase B tests: recovery report persisted on rollback ack, pre-existing-critical report stored without blaming the update, absent recovery key leaves it null, non-array recovery rejected 422.

Dashboard — npx tsc --noEmit clean; npm run build succeeds.

11. Live-site verification (must be done on a staging WordPress)

These cannot be exercised on the build VM (no running WordPress) and must be confirmed on a staging site:

1. Install connector 1.2.11; confirm mu-plugins/marqira-guard.php is auto-copied into wp-content/mu-plugins/ and appears under **Plugins** → **Must-Use**.
 2. Trigger a plugin update to a deliberately fatal version → confirm the site is detected critical, the plugin is rolled back/deactivated, the site recovers, and the dashboard shows the *rolled back & recovered* banner.
 3. Break the site *before* issuing an update → confirm the ack reports `pre_existing_critical` and nothing is rolled back.
 4. Confirm GET `/marqira/v1/health-ping` returns 200 on a healthy site.
 5. Confirm the mu-plugin guard fires when a fatal happens mid-update while the connector plugin itself cannot load, and that only the sentinel's named plugin is deactivated.
-

12. Deployment

No API config changes beyond running migrations. See COOLIFY.md → *Phase B* for the exact deploy steps.